

Java Backend Architecture

Interview Series

Chapter 13.1

Synchronous vs Asynchronous vs Semi-Sync

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Introduction

Synchronous and asynchronous programming models are foundational to understanding how modern Java backend systems handle concurrency and I/O. In a synchronous model, the calling thread blocks until the operation completes — simple to reason about, but wasteful under high I/O load. Asynchronous models free the thread immediately, delivering the result later via a callback, Future, or reactive stream. Java has evolved dramatically in this space: from raw threads and Callable/Future in Java 5, through CompletableFuture in Java 8, to virtual threads in Java 21 that make blocking code scale like async.

Semi-synchronous designs split the difference: the caller submits work to a thread pool asynchronously but then waits (blocks) on the result — common in servlet containers like Tomcat. The C10K problem (handling 10,000 concurrent connections) drove the adoption of event-loop architectures (Netty, Node.js, Spring WebFlux) which multiplex many requests onto a small, fixed set of threads. Senior engineers must understand the throughput implications of each model, Amdahl's law limits on parallelism, and when virtual threads finally make thread-per-request viable at massive scale.

Key Concepts & Definitions

Synchronous	Calling thread blocks until the I/O or computation finishes; simple but CPU-wasteful under high I/O.
Asynchronous	Calling thread is freed immediately; result delivered later via callback, Future, or reactive stream.
Semi-Synchronous	Request dispatched to a worker pool asynchronously; caller then blocks waiting for the pooled result.
CompletableFuture	Java 8 non-blocking future supporting rich chaining (.thenApply, .thenCompose, .exceptionally, .allOf).
@Async	Spring annotation that runs a method in a separate thread pool, returning CompletableFuture or void.
Virtual Thread	Java 21 lightweight thread managed by JVM, not OS. Millions can exist; blocking calls are parked not blocked.
Event Loop	Single-threaded loop that dispatches I/O events; used by Netty, Node.js, WebFlux. No thread-per-request.
Thread-per-Request	Each HTTP request gets its own OS thread (Tomcat default); limits concurrency to pool size.
C10K Problem	Challenge of handling 10,000 concurrent connections with traditional blocking thread models.
Amdahl's Law	Maximum speedup limited by serial fraction: $\text{Speedup} = 1 / (S + (1-S)/N)$ where S is serial proportion.
ForkJoinPool	Work-stealing thread pool used by CompletableFuture by default and parallel streams.
thenCompose	CompletableFuture method for flat-mapping; returns a new CompletableFuture from a function returning one.
thenApply	Transforms CompletableFuture result synchronously; analogous to map() on a stream.

exceptionally	CompletableFuture error handler; provides fallback value when upstream stage fails.
Platform Thread	Traditional Java thread mapped 1:1 to an OS thread; expensive to create, limited by OS scheduler.

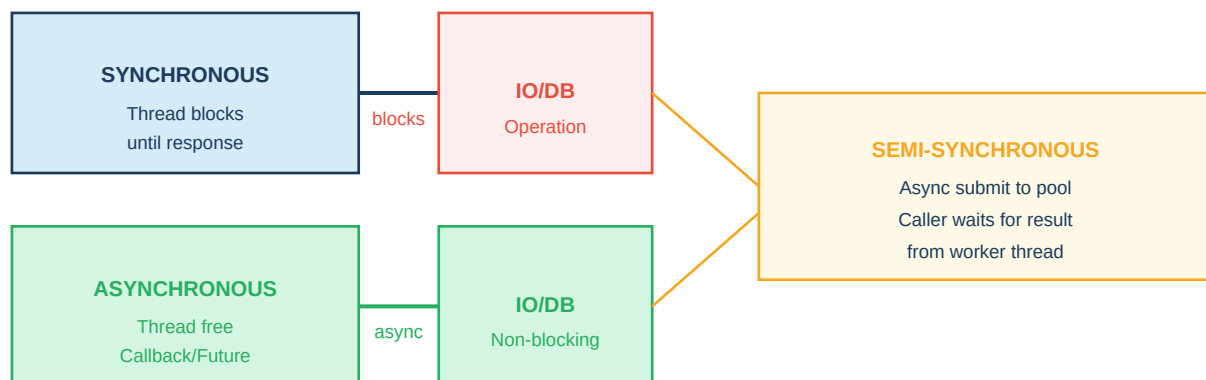
Model Comparison

Model	Thread Usage	Throughput	Complexity	Best For
Synchronous	Blocked per req	Low (I/O bound)	Low	Simple CRUD, low load
Async CF	ForkJoinPool	High	Medium	CPU + I/O parallel work
Semi-Sync	Pool + caller wait	Medium	Low-Med	Servlet containers
Event Loop	Fixed small pool	Very High	High	High concurrency I/O
Virtual Threads	JVM managed	Very High	Low	Blocking code at scale
@Async Spring	Custom executor	Medium-High	Low	Spring service offload

Architecture Diagrams

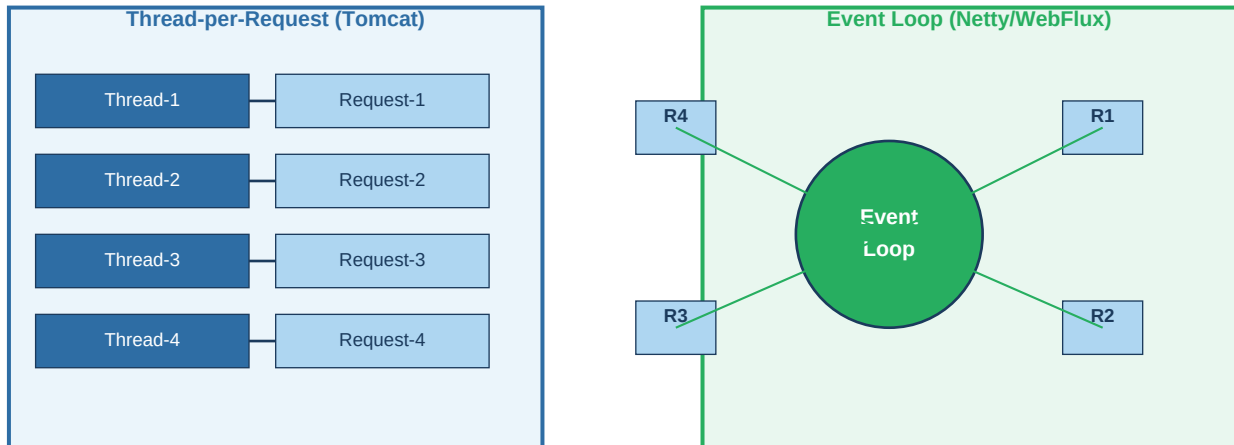
Synchronous vs Asynchronous vs Semi-Synchronous

Threading Models Comparison



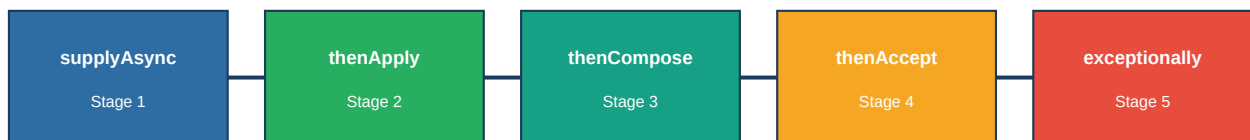
Thread-per-Request vs Event Loop

Thread-per-Request vs Event Loop



CompletableFuture Chaining Pipeline

CompletableFuture Chaining Pipeline



Each stage runs on ForkJoinPool unless executor provided

Java Code Examples

1. Basic Synchronous vs Asynchronous Call

```
// Synchronous - thread blocks String result = restTemplate.getForObject(url, String.class); //
blocks here // Asynchronous with CompletableFuture CompletableFuture<String> future =
CompletableFuture.supplyAsync(() -> httpClient.send(request, BodyHandlers.ofString()).body()
); future.thenAccept(body -> System.out.println("Got: " + body)); // main thread continues
immediately
```

2. CompletableFuture Chaining

```
CompletableFuture<OrderResponse> pipeline = CompletableFuture.supplyAsync(() ->
fetchUser(userId)) // async .thenApply(user -> enrichWithProfile(user)) // transform
.thenCompose(user -> placeOrderAsync(user, items)) // flat-map .thenApply(order ->
buildResponse(order)) .exceptionally(ex -> { log.error("Pipeline failed", ex); return
OrderResponse.error(ex.getMessage()); }); OrderResponse resp = pipeline.get(5,
TimeUnit.SECONDS); // wait max 5s
```

3. Spring @Async with Custom Executor

```
@Configuration @EnableAsync public class AsyncConfig { @Bean("reportExecutor") public Executor
reportExecutor() { ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
exec.setCorePoolSize(4); exec.setMaxPoolSize(10); exec.setQueueCapacity(50);
exec.setThreadNamePrefix("report-"); exec.initialize(); return exec; } } @Service public class
ReportService { @Async("reportExecutor") public CompletableFuture<Report> generateReport(Long
id) { // runs in reportExecutor pool, not caller thread return
```

```
CompletableFuture.completedFuture(buildReport(id)); } }
```

4. Virtual Threads (Java 21)

```
// Java 21 - virtual thread executor try (var executor =
Executors.newVirtualThreadPerTaskExecutor()) { List<Future<String>> futures =
IntStream.range(0, 10_000) .mapToObj(i -> executor.submit(() -> blockingHttpCall(i)))
.toList(); for (Future<String> f : futures) { System.out.println(f.get()); // each call blocks
a virtual thread, not an OS thread } } // Spring Boot 3.2+ - enable virtual threads in
application.properties: // spring.threads.virtual.enabled=true
```

5. CompletableFuture.allOf for Fan-Out/Fan-In

```
CompletableFuture<UserProfile> userF = fetchUserAsync(id); CompletableFuture<List<Order>>
ordersF = fetchOrdersAsync(id); CompletableFuture<CreditScore> creditF = fetchCreditAsync(id);
CompletableFuture<DashboardData> dashboard = CompletableFuture.allOf(userF, ordersF, creditF)
.thenApply(v -> new DashboardData( userF.join(), ordersF.join(), creditF.join() )); // All 3
calls run in parallel; allOf completes when last one finishes
```

6. Event Loop with Netty (Pseudo-Code)

```
// Netty NIO - single event loop handles many connections EventLoopGroup bossGroup = new
NioEventLoopGroup(1); // accept connections EventLoopGroup workerGroup = new
NioEventLoopGroup(4); // handle I/O ServerBootstrap b = new ServerBootstrap();
b.group(bossGroup, workerGroup) .channel(NioServerSocketChannel.class) .childHandler(new
ChannelInitializer<SocketChannel>() { protected void initChannel(SocketChannel ch) {
ch.pipeline().addLast(new HttpServerCodec(), new MyHandler()); } }); //
MyHandler.channelRead() is called by worker threads - NEVER block here! // Offload blocking
work to a separate executor pool
```

Interview Q&A;

Q1. What is the key difference between synchronous and asynchronous programming?

In synchronous programming the calling thread blocks until the operation completes, wasting CPU cycles waiting for I/O. In asynchronous programming the thread is freed immediately and a callback, `CompletableFuture`, or reactive stream delivers the result later, allowing the thread to serve other requests in the meantime.

Q2. What is a `CompletableFuture` and how does it differ from a plain `Future`?

`CompletableFuture` (Java 8) extends `Future` with a rich, non-blocking chaining API (`.thenApply`, `.thenCompose`, `.thenCombine`, `.exceptionally`, `.allOf`, `.anyOf`). Plain `Future` only allows blocking `get()` with no composition support. `CompletableFuture` can also be completed explicitly (`complete()`, `completeExceptionally()`).

Q3. Explain the C10K problem and how event loops solve it.

C10K is the challenge of handling 10,000 concurrent connections with traditional thread-per-connection models, which require 10,000 OS threads consuming GBs of stack memory. Event loops like Netty multiplex all connections onto a tiny fixed pool (e.g., `CPU_COUNT` threads) using non-blocking NIO selectors, requiring no extra threads for idle connections.

Q4. What is Amdahl's Law and why does it matter for async systems?

Amdahl's Law states that the maximum speedup from parallelisation is $1/(S + (1-S)/N)$ where S is the serial fraction and N is the number of processors. Even with infinite threads, serial portions cap speedup. For async systems this means I/O parallelism has great returns, but CPU-bound serial logic remains the bottleneck.

Q5. What is a semi-synchronous pattern?

A semi-synchronous pattern dispatches the request asynchronously to a worker pool (the submission is non-blocking) but then the caller blocks waiting for the result. Tomcat's servlet model is semi-synchronous: the HTTP connector posts work to an executor, and the calling thread waits for the servlet to finish before sending the response.

Q6. How does @Async work in Spring and what are its pitfalls?

@Async proxies the annotated method to run in a separate thread pool. Pitfalls: (1) calling @Async from within the same class bypasses the proxy and runs synchronously; (2) checked exceptions are swallowed unless the return type is CompletableFuture; (3) using the default SimpleAsyncTaskExecutor creates a new thread per call — always configure a custom pool.

Q7. What are virtual threads (Project Loom) and how do they help?

Virtual threads are JVM-managed, lightweight threads (not OS threads). They can be created by the millions. When a virtual thread blocks on I/O, the JVM parks it and reuses the underlying OS carrier thread for other work. This makes blocking code scale like async code without rewriting to callbacks or reactive streams. Available since Java 21.

Q8. When should you use CompletableFuture vs reactive streams?

CompletableFuture is ideal for one-off async computations, fan-out/fan-in, and parallel service calls. Reactive streams (Reactor, RxJava) are better when you need to process data streams with backpressure, compose complex operator pipelines, or integrate with reactive libraries (R2DBC, WebClient). Reactive adds complexity; prefer CompletableFuture for simpler cases.

Q9. What does .thenCompose() do and when do you use it over .thenApply()?

.thenApply() is like map — it transforms the value synchronously and returns CompletableFuture<U>. .thenCompose() is like flatMap — it returns a function that itself returns a CompletableFuture<U>, flattening to avoid CompletableFuture. Use thenCompose when the next step is itself asynchronous.

Q10. How does the ForkJoinPool relate to CompletableFuture?

By default, CompletableFuture.supplyAsync() and other async stages run on ForkJoinPool.commonPool(). This pool has (CPU_COUNT - 1) threads. For I/O-heavy tasks this is too small; always provide a custom Executor to avoid starving CPU-bound work or blocking the common pool.

Q11. Explain the thread-per-request model in Tomcat.

Tomcat maintains a thread pool (default 200 threads). Each incoming HTTP request is assigned a thread from the pool for its entire lifecycle — reading the request, executing business logic, writing the response. If all 200 threads are busy, new requests queue or get rejected. This limits concurrency to the pool size, which is why event-loop servers outperform Tomcat under high I/O concurrency.

Q12. What is the difference between thenApply and thenApplyAsync?

thenApply() executes the function on the thread that completed the previous stage (possibly the caller). thenApplyAsync() executes on a thread from the supplied executor (or common pool). Use the Async variant when the function is slow or blocking to avoid holding up the completing thread.

Q13. How do you handle exceptions in a CompletableFuture chain?

Use .exceptionally(ex -> fallback) for a simple fallback value. Use .handle((result, ex) -> ...) to inspect both result and exception at any stage. Use .whenComplete((result, ex) -> ...) for side-effects without transforming. Always handle exceptions; an unhandled CF exception is silently swallowed unless you call .get() or join().

Q14. What is the role of Netty in Spring WebFlux?

Spring WebFlux defaults to Netty as its embedded server. Netty provides a non-blocking, event-driven NIO server with boss/worker EventLoopGroups. WebFlux maps reactive Flux/Mono pipelines onto Netty's event loop, ensuring no OS thread blocks on I/O. The entire request handling chain stays reactive from socket to response.

Q15. How do virtual threads interact with synchronized blocks?

In Java 21, a virtual thread that enters a synchronized block pins its carrier OS thread — it cannot unmount and let the carrier serve another virtual thread. This "pinning" can become a bottleneck if synchronized blocks contain blocking I/O. The fix is to replace synchronized with ReentrantLock (which supports virtual thread unmounting on park).

Q16. What is the difference between parallelism and concurrency?

Concurrency is about structuring a program to handle multiple tasks simultaneously (interleaving on one or more CPUs). Parallelism is about executing tasks physically at the same time on multiple CPUs. Async/event-loop models achieve high concurrency with low parallelism; virtual threads achieve high concurrency with blocking code; ForkJoin achieves parallel CPU computation.

Q17. How do you configure a custom thread pool for @Async in Spring Boot?

Create a @Bean of type ThreadPoolTaskExecutor or ThreadPoolExecutor with desired core/max/queue settings and pass its name to @Async("beanName"). Register AsyncConfigurer to set a default executor. Always set thread name prefix for observability, and configure a RejectedExecutionHandler for overflow.

Q18. What happens when CompletableFuture.allOf() has one failing stage?

allOf() completes exceptionally as soon as any stage fails (in Java 8+). The other stages continue running but their results are ignored unless you joined them separately. The returned CF carries the first exception. Use .handle() on the allOf() future to process partial failures gracefully.

Q19. Explain the concept of back-pressure in async systems.

Backpressure is a flow-control mechanism where a slow consumer signals to a fast producer to slow down, preventing unbounded queue growth and OOM errors. Plain CompletableFuture has no backpressure; reactive streams (Flux/Mono) implement the Reactive Streams spec which includes a request(n) protocol for demand signalling.

Q20. When should you NOT use async/reactive programming?

Avoid async for CPU-bound, purely computational work where synchronous sequential code is simpler and equally fast. Avoid reactive when the team lacks experience — debugging stack traces are difficult. For simple CRUD with low concurrency, synchronous Spring MVC + virtual threads (Java 21) is easier to maintain. Premature asyncification adds complexity without proportional benefit.